

Notes on Chapter 26 - Iteration

The Caveman Coder

2025-08-16

This chapter introduces the concept of **functional programming**.

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Situation: We are tasked to count the number of observations and compute the median in the tibble below:

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
```

This can be solved using copy-paste:

```
df |> summarize(
  n = n(),
  a = median(a),
  b = median(b),
  c = median(c),
  d = median(d)
)
```

```
## # A tibble: 1 x 5
##       n         a         b         c         d
##   <int>   <dbl>   <dbl>   <dbl>   <dbl>
## 1     10 -0.0599 -0.209  -0.191  0.211
```

This breaks the rule of never copy-pasting more than twice, and is prone to mistakes.

Using `across()` to solve the problem above:

```
df |> summarize(
  n = n(),
  across(a:d, median)
)
```

```
## # A tibble: 1 x 5
##       n         a         b         c         d
##   <int>   <dbl>   <dbl>   <dbl>   <dbl>
## 1     10 -0.0599 -0.209 -0.191  0.211
```

Voila!

Selecting columns with .cols

- As the first argument to `across()`, `.cols` specifies which columns you want to iterate over.
- The second argument `.fn` specifies what to do with each column.

Example using `across()` with `everything()`:

```
df <- tibble(
  grp = sample(2, 10, replace = TRUE),
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)

df |>
  group_by(grp) |>
  summarize(across(everything(), median))
```

```
## # A tibble: 2 x 5
##   grp         a         b         c         d
##   <int>   <dbl>   <dbl>   <dbl>   <dbl>
## 1     1 -0.642 -0.245  0.419  0.500
## 2     2 -0.375  0.434 -0.0653 -0.237
```

`where()` allows the user to select columns based on type:

- `where(is.numeric)` selects numeric columns.
- `where(is.character)` selects all string columns.
- `where(is.Date)` selects all date columns.
- `where(is.POSIXct)` selects all date-time columns.
- `where(is.logical)` selects all logical columns.

Calling a single function

Remember, do not put `()` after the function that you are passing to `across()`. The code below will throw an error:

```
df |>
  group_by(grp) |>
  summarize(across(everything(), median()))
```

What if we want to pass an argument to the function we are passing to `across()`? For example, the argument `na.rm = TRUE` on `median()`.

Solution: instead of calling the target function directly, we create a new function that calls our target function.

Example:

```
rnorm_na <- function(n, n_na, mean = 0, sd = 1) {
  sample(c(rnorm(n - n_na, mean = mean, sd = sd), rep(NA, n_na)))
}

df_miss <- tibble(
  a = rnorm_na(5, 1),
  b = rnorm_na(5, 1),
  c = rnorm_na(5, 2),
  d = rnorm(5)
)

# This is the solution
df_miss |>
  summarize(
    across(a:d, function(x) median(x, na.rm = TRUE)),
    n = n()
  )
```

```
## # A tibble: 1 x 5
##       a         b         c         d         n
##   <dbl>   <dbl> <dbl> <dbl> <int>
## 1 -0.698 -0.00535 0.365 0.537     5
```

There is shortcut for this “throwaway” function: the `\`. Just replace the word `function` with `\`.

Example of handling the anonymous or throwaway function using `\`:

```
df_miss |>
  summarize(
    across(a:d, \(x) median(x, na.rm = TRUE)),
    n = n()
  )
```

```
## # A tibble: 1 x 5
##       a         b         c         d         n
##   <dbl>   <dbl> <dbl> <dbl> <int>
## 1 -0.698 -0.00535 0.365 0.537     5
```

How to know how many values were removed when we specified `na.rm = TRUE`? By supplying another function to `across()`:

```
df_miss |>
  summarize(
    across(a:d, list(
      median = \(x) median(x, na.rm = TRUE),
      n_miss = \(x) sum(is.na(x))
    )),
    n = n()
  )
```

```
## # A tibble: 1 x 9
##   a_median a_n_miss b_median b_n_miss c_median c_n_miss d_median d_n_miss     n
##   <dbl>   <int>   <dbl>   <int>   <dbl>   <int>   <dbl>   <int> <int>
## 1  -0.698     1 -0.00535     1   0.365     2   0.537     0     5
```

Column names

Specifying what to name the results using the `.names` argument:

```
df_miss |>
  summarize(
    across(
      a:d,
      list(
        median = \(x) median(x, na.rm = TRUE),
        n_miss = \(x) sum(is.na(x))
      ),
      .names = "{.fn}_{.col}"
    ),
    n = n()
  )
```

```
## # A tibble: 1 x 9
##   median_a n_miss_a median_b n_miss_b median_c n_miss_c median_d n_miss_d     n
##   <dbl>   <int>   <dbl>   <int>   <dbl>   <int>   <dbl>   <int> <int>
## 1  -0.698     1 -0.00535     1   0.365     2   0.537     0     5
```

When using `across()` with `mutate()`, the default action is for `across()` to use the same names as inputs. This means that if you don't use the `.names` argument, the original values of the columns will be replaced.

Example using `coalesce()` to replace NAs with 0:

```
df_miss |>
  mutate(
    across(a:d, \(x) coalesce(x, 0))
  )
```

```
## # A tibble: 5 x 4
##       a       b       c       d
##   <dbl> <dbl> <dbl> <dbl>
## 1 -0.908 -0.239 -0.936 -1.64
## 2 -0.488  0.228  0.783  1.62
## 3  0      -0.401  0.365 -1.79
## 4 -1.32   0       0      0.537
## 5 -0.185  0.773  0      1.04
```

If this is not your intended action, and would instead prefer to create new columns, we can specify the naming format for the new names using `.names`:

```
df_miss |>
  mutate(
    across(a:d, \(x) coalesce(x, 0), .names = "{.col}_na_zero")
  )
```

```
## # A tibble: 5 x 8
##       a       b       c       d a_na_zero b_na_zero c_na_zero d_na_zero
##   <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 -0.908 -0.239 -0.936 -1.64   -0.908   -0.239   -0.936   -1.64
```

```
## 2 -0.488  0.228  0.783  1.62    -0.488    0.228    0.783    1.62
## 3 NA      -0.401  0.365 -1.79      0      -0.401    0.365   -1.79
## 4 -1.32   NA      NA      0.537   -1.32     0        0      0.537
## 5 -0.185  0.773 NA      1.04    -0.185    0.773    0      1.04
```

Filtering

For filtering, `if_any()` and `if_all()` are the variants of `across()` provided by `dplyr`.

Example:

```
df_miss |> filter(if_any(a:d, is.na))
```

```
## # A tibble: 3 x 4
##       a         b         c         d
##   <dbl> <dbl> <dbl> <dbl>
## 1 NA     -0.401  0.365 -1.79
## 2 -1.32   NA      NA      0.537
## 3 -0.185  0.773 NA      1.04
```

```
df_miss |> filter(if_all(a:d, is.na))
```

```
## # A tibble: 0 x 4
## # i 4 variables: a <dbl>, b <dbl>, c <dbl>, d <dbl>
```

`across()` in functions

Date expander from Jacob Scott:

```
expand_dates <- function(df) {
  df |>
    mutate(
      across(where(is.Date), list(
        year = year,
        month = month,
        dat = mday
      ))
    )
}

df_date <- tibble(
  name = c("Amy", "Bob"),
  date = ymd(c("2009-08-03", "2010-01-16"))
)

df_date |>
  expand_dates()
```

```
## # A tibble: 2 x 5
##   name  date      date_year date_month date_dat
##   <chr> <date>      <dbl>      <dbl>      <int>
## 1 Amy   2009-08-03    2009         8         3
## 2 Bob   2010-01-16    2010         1        16
```

Example computing the means of numeric columns:

```
summarize_means <- function(df, summary_vars = where(is.numeric)) {
  df |>
```

```

    summarize(
      across({ summary_vars }, \(x) mean(x, na.rm = TRUE)),
      n = n(),
      .groups = "drop"
    )
}

```

```

diamonds |>
  group_by(cut) |>
  summarize_means()

```

```

## # A tibble: 5 x 9
##   cut      carat depth table price      x      y      z      n
##   <ord>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <int>
## 1 Fair      1.05  64.0  59.1 4359.  6.25  6.18  3.98 1610
## 2 Good      0.849  62.4  58.7 3929.  5.84  5.85  3.64 4906
## 3 Very Good 0.806  61.8  58.0 3982.  5.74  5.77  3.56 12082
## 4 Premium   0.892  61.3  58.7 4584.  5.97  5.94  3.65 13791
## 5 Ideal     0.703  61.7  56.0 3458.  5.51  5.52  3.40 21551

```

```

diamonds |>
  group_by(cut) |>
  summarize_means(c(carat, x:z))

```

```

## # A tibble: 5 x 6
##   cut      carat      x      y      z      n
##   <ord>    <dbl> <dbl> <dbl> <dbl> <int>
## 1 Fair      1.05  6.25  6.18  3.98 1610
## 2 Good      0.849  5.84  5.85  3.64 4906
## 3 Very Good 0.806  5.74  5.77  3.56 12082
## 4 Premium   0.892  5.97  5.94  3.65 13791
## 5 Ideal     0.703  5.51  5.52  3.40 21551

```

Compare with pivot_longer()

Example of code using across() and summarize() to do “pivot” operations:

```

df |>
  summarize(across(a:d, list(median = median, mean = mean)))

## # A tibble: 1 x 8
##   a_median a_mean b_median b_mean c_median c_mean d_median d_mean
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  -0.504 -0.503 -0.0380 0.0417  0.239  0.395  0.217  0.178

```

We could compute the same values using pivot_longer() and summarize():

```

long <- df |>
  pivot_longer(a:d) |>
  group_by(name) |>
  summarize(
    median = median(value),
    mean = mean(value)
  )

long

```

```
## # A tibble: 4 x 3
##   name    median    mean
##   <chr>   <dbl>   <dbl>
## 1 a      -0.504 -0.503
## 2 b      -0.0380 0.0417
## 3 c       0.239 0.395
## 4 d       0.217 0.178
```

To make the format similar to the first result, we could pivot again:

```
long |>
  pivot_wider(
    names_from = name,
    values_from = c(median, mean),
    names_vary = "slowest",
    names_glue = "{name}_{.value}"
  )
```

```
## # A tibble: 1 x 8
##   a_median a_mean b_median b_mean c_median c_mean d_median d_mean
##   <dbl> <dbl>   <dbl> <dbl>   <dbl> <dbl>   <dbl> <dbl>
## 1  -0.504 -0.503  -0.0380 0.0417    0.239 0.395    0.217 0.178
```

Using this technique will enable us to do more complicated operations that is not directly possible with `across()`. For example, consider the data below:

```
df_paired <- tibble(
  a_val = rnorm(10),
  a_wts = runif(10),
  b_val = rnorm(10),
  b_wts = runif(10),
  c_val = rnorm(10),
  c_wts = runif(10),
  d_val = rnorm(10),
  d_wts = runif(10)
)
```

```
df_paired
```

```
## # A tibble: 10 x 8
##   a_val a_wts b_val b_wts c_val c_wts d_val d_wts
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 -0.762 0.168 1.11 0.116 -1.78 0.154 -0.206 0.538
## 2 1.53 0.336 2.13 0.00141 -1.67 0.370 0.953 0.680
## 3 0.718 0.587 1.69 0.705 -0.932 0.384 0.0430 0.985
## 4 1.74 0.550 0.0191 0.0115 -0.388 0.374 -0.530 0.320
## 5 -0.997 0.155 -1.02 0.322 0.622 0.411 -0.381 0.374
## 6 -1.76 0.972 -1.21 0.933 -0.331 0.516 0.664 0.285
## 7 -1.62 0.446 1.91 0.763 0.536 0.945 -0.533 0.174
## 8 -0.458 0.636 0.204 0.608 -1.40 0.810 -0.812 0.775
## 9 1.21 0.271 0.0703 0.383 -1.31 0.308 -0.0732 0.774
## 10 0.0202 0.393 1.35 0.117 1.50 0.399 -0.167 0.530
```

Suppose we want to compute the weighted means, there is currently no way to do this with `across()`, but this could be solved with `pivot_longer()`:

```
df_long <- df_paired |>
  pivot_longer(
    everything(),
    names_to = c("group", ".value"),
    names_sep = "_"
  )
```

```
df_long
```

```
## # A tibble: 40 x 3
##   group    val    wts
##   <chr> <dbl> <dbl>
## 1 a     -0.762 0.168
## 2 b      1.11 0.116
## 3 c     -1.78 0.154
## 4 d     -0.206 0.538
## 5 a      1.53 0.336
## 6 b      2.13 0.00141
## 7 c     -1.67 0.370
## 8 d      0.953 0.680
## 9 a      0.718 0.587
## 10 b     1.69 0.705
## # i 30 more rows
```

```
df_long |>
  group_by(group) |>
  summarize(mean = weighted.mean(val, wts))
```

```
## # A tibble: 4 x 2
##   group    mean
##   <chr> <dbl>
## 1 a     -0.172
## 2 b      0.414
## 3 c     -0.372
## 4 d     -0.0756
```

Similar to the example before, we could use `pivot_wider()` if needed.

Exercises

xxxxxxx

Reading multiple files

Listing files in a directory

Syntax is `list.files(path, pattern, full.names = TRUE)` where:

- `path` is the directory to look in.
- `pattern` is the regex used to filter the names. The most common pattern is `[.]xlsx$` or `[.]csv$` to find the files by specifying the extension name.
- if `full.names` is set to `TRUE`, the full directory name will be included in the output.

Example:

Listing all Excel files in the specified directory:


```
paths <- list.files("data/gapminder", pattern = "[.]xlsx$", full.names = TRUE)
```

```
paths
```

```
## [1] "data/gapminder/1952.xlsx" "data/gapminder/1957.xlsx"
## [3] "data/gapminder/1962.xlsx" "data/gapminder/1967.xlsx"
## [5] "data/gapminder/1972.xlsx" "data/gapminder/1977.xlsx"
## [7] "data/gapminder/1982.xlsx" "data/gapminder/1987.xlsx"
## [9] "data/gapminder/1992.xlsx" "data/gapminder/1997.xlsx"
## [11] "data/gapminder/2002.xlsx" "data/gapminder/2007.xlsx"
```

Lists

Creating a list of Excel files by reading the files specified in the path names:

```
files <- list(
  readxl::read_excel("data/gapminder/1952.xlsx"),
  readxl::read_excel("data/gapminder/1957.xlsx"),
  readxl::read_excel("data/gapminder/1962.xlsx"),
  readxl::read_excel("data/gapminder/1967.xlsx"),
  readxl::read_excel("data/gapminder/1972.xlsx"),
  readxl::read_excel("data/gapminder/1977.xlsx"),
  readxl::read_excel("data/gapminder/1982.xlsx"),
  readxl::read_excel("data/gapminder/1987.xlsx"),
  readxl::read_excel("data/gapminder/1992.xlsx"),
  readxl::read_excel("data/gapminder/1992.xlsx"),
  readxl::read_excel("data/gapminder/2002.xlsx"),
  readxl::read_excel("data/gapminder/2007.xlsx")
)
```

Accessing the target file from the list of files:

```
files[[3]]
```

```
## # A tibble: 142 x 6
##   country    continent  year lifeExp    pop gdpPercap
##   <chr>      <chr>    <dbl> <dbl>    <dbl> <dbl>
## 1 Afghanistan Asia      1962  32.0 10267083    853.
## 2 Albania    Europe   1962  64.8 1728137    2313.
## 3 Algeria    Africa   1962  48.3 11000948   2551.
## 4 Angola     Africa   1962   34  4826015   4269.
## 5 Argentina  Americas 1962  65.1 21283783   7133.
## 6 Australia  Oceania   1962  70.9 10794968  12217.
## 7 Austria    Europe   1962  69.5  7129864  10751.
## 8 Bahrain    Asia     1962  56.9   171863  12753.
## 9 Bangladesh Asia     1962  41.2 56839289    686.
## 10 Belgium   Europe   1962  70.2  9218400  10991.
## # i 132 more rows
```

`purrr::map()` and `list_rbind()`

If collecting the files by hand is tedious, this method provides a smarter way of doing things:

Using `map()` to get a list of data frames:

```
files <- map(paths, readxl::read_excel)
length(files)
```

```
## [1] 12
```

```
# test
files[[3]]
```

```
## # A tibble: 142 x 6
##   country    continent year lifeExp      pop gdpPercap
##   <chr>      <chr>    <dbl>  <dbl>    <dbl>    <dbl>
## 1 Afghanistan Asia      1962   32.0 10267083    853.
## 2 Albania    Europe    1962   64.8  1728137   2313.
## 3 Algeria    Africa    1962   48.3 11000948   2551.
## 4 Angola     Africa    1962    34   4826015   4269.
## 5 Argentina  Americas  1962   65.1 21283783   7133.
## 6 Australia  Oceania   1962   70.9 10794968  12217.
## 7 Austria    Europe    1962   69.5  7129864  10751.
## 8 Bahrain    Asia      1962   56.9   171863  12753.
## 9 Bangladesh Asia      1962   41.2 56839289    686.
## 10 Belgium   Europe    1962   70.2  9218400  10991.
## # i 132 more rows
```

Combining all the files into a single file using `purrr::list_rbind()`:

```
list_rbind(files)
```

```
## # A tibble: 1,704 x 6
##   country    continent year lifeExp      pop gdpPercap
##   <chr>      <chr>    <dbl>  <dbl>    <dbl>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Albania    Europe    1952   55.2  1282697   1601.
## 3 Algeria    Africa    1952   43.1  9279525   2449.
## 4 Angola     Africa    1952   30.0  4232095   3521.
## 5 Argentina  Americas  1952   62.5 17876956   5911.
## 6 Australia  Oceania   1952   69.1  8691212  10040.
## 7 Austria    Europe    1952   66.8  6927772   6137.
## 8 Bahrain    Asia      1952   50.9   120447   9867.
## 9 Bangladesh Asia      1952   37.5 46886859    684.
## 10 Belgium   Europe    1952    68   8730405   8343.
## # i 1,694 more rows
```

Doing the steps above in a pipeline:

```
paths |>
  map(readxl::read_excel) |>
  list_rbind()
```

```
## # A tibble: 1,704 x 6
##   country    continent year lifeExp      pop gdpPercap
##   <chr>      <chr>    <dbl>  <dbl>    <dbl>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Albania    Europe    1952   55.2  1282697   1601.
## 3 Algeria    Africa    1952   43.1  9279525   2449.
## 4 Angola     Africa    1952   30.0  4232095   3521.
## 5 Argentina  Americas  1952   62.5 17876956   5911.
## 6 Australia  Oceania   1952   69.1  8691212  10040.
## 7 Austria    Europe    1952   66.8  6927772   6137.
## 8 Bahrain    Asia      1952   50.9   120447   9867.
## 9 Bangladesh Asia      1952   37.5 46886859    684.
```

```
## 10 Belgium      Europe      1952      68      8730405      8343.
## # i 1,694 more rows
```

In case it is necessary to pass more arguments to `read_excel()`, the technique for handling anonymous functions (aka lambda functions) can also be used here:

```
paths |>
  map(\(path) readxl::read_excel(path, n_max = 1)) |>
  list_rbind()
```

```
## # A tibble: 12 x 6
##   country      continent year lifeExp      pop gdpPercap
##   <chr>         <chr>    <dbl>  <dbl>    <dbl>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Afghanistan Asia      1957   30.3  9240934    821.
## 3 Afghanistan Asia      1962   32.0 10267083    853.
## 4 Afghanistan Asia      1967   34.0 11537966    836.
## 5 Afghanistan Asia      1972   36.1 13079460    740.
## 6 Afghanistan Asia      1977   38.4 14880372    786.
## 7 Afghanistan Asia      1982   39.9 12881816    978.
## 8 Afghanistan Asia      1987   40.8 13867957    852.
## 9 Afghanistan Asia      1992   41.7 16317921    649.
## 10 Afghanistan Asia      1997   41.8 22227415    635.
## 11 Afghanistan Asia      2002   42.1 25268405    727.
## 12 Afghanistan Asia      2007   43.8 31889923    975.
```

Data in the path

Extracting the filename from the full path using `set_names(basename)`:

```
paths |> set_names(basename)

##           1952.xlsx           1957.xlsx
## "data/gapminder/1952.xlsx" "data/gapminder/1957.xlsx"
##           1962.xlsx           1967.xlsx
## "data/gapminder/1962.xlsx" "data/gapminder/1967.xlsx"
##           1972.xlsx           1977.xlsx
## "data/gapminder/1972.xlsx" "data/gapminder/1977.xlsx"
##           1982.xlsx           1987.xlsx
## "data/gapminder/1982.xlsx" "data/gapminder/1987.xlsx"
##           1992.xlsx           1997.xlsx
## "data/gapminder/1992.xlsx" "data/gapminder/1997.xlsx"
##           2002.xlsx           2007.xlsx
## "data/gapminder/2002.xlsx" "data/gapminder/2007.xlsx"
```

These set names are automatically carried along by all the map functions, allowing us to do this:

```
files <- paths |>
  set_names(basename) |>
  map(readxl::read_excel)

files[["1962.xlsx"]]
```

```
## # A tibble: 142 x 6
##   country      continent year lifeExp      pop gdpPercap
##   <chr>         <chr>    <dbl>  <dbl>    <dbl>    <dbl>
## 1 Afghanistan Asia      1962   32.0 10267083    853.
## 2 Albania      Europe    1962   64.8 1728137    2313.
```

```
## 3 Algeria      Africa      1962      48.3 11000948      2551.
## 4 Angola       Africa      1962      34   4826015      4269.
## 5 Argentina    Americas    1962      65.1 21283783      7133.
## 6 Australia    Oceania     1962      70.9 10794968      12217.
## 7 Austria      Europe      1962      69.5 7129864       10751.
## 8 Bahrain      Asia        1962      56.9 171863        12753.
## 9 Bangladesh   Asia        1962      41.2 56839289       686.
## 10 Belgium     Europe      1962      70.2 9218400        10991.
## # i 132 more rows
```

Writing the filenames into a new column during rbind:

```
paths |>
  set_names(basename) |>
  map(readxl::read_excel) |>
  list_rbind(names_to = "source")

## # A tibble: 1,704 x 7
##   source      country      continent  year lifeExp      pop gdpPercap
##   <chr>      <chr>      <chr>      <dbl> <dbl>      <dbl> <dbl>
## 1 1952.xlsx Afghanistan Asia      1952    28.8 8425333    779.
## 2 1952.xlsx Albania     Europe    1952    55.2 1282697    1601.
## 3 1952.xlsx Algeria     Africa    1952    43.1 9279525    2449.
## 4 1952.xlsx Angola      Africa    1952    30.0 4232095    3521.
## 5 1952.xlsx Argentina   Americas  1952    62.5 17876956    5911.
## 6 1952.xlsx Australia   Oceania   1952    69.1 8691212    10040.
## 7 1952.xlsx Austria     Europe    1952    66.8 6927772    6137.
## 8 1952.xlsx Bahrain     Asia      1952    50.9 120447     9867.
## 9 1952.xlsx Bangladesh   Asia      1952    37.5 46886859    684.
## 10 1952.xlsx Belgium    Europe    1952    68   8730405    8343.
## # i 1,694 more rows
```

If the filepath contains important information, we can include them in the data like this:

```
paths |>
  set_names() |>
  map(readxl::read_excel) |>
  list_rbind(names_to = "source") |>
  separate_wider_delim(source, delim = "/", names = c(NA, "dir", "file")) |>
  separate_wider_delim(file, delim = ".", names = c("name", "ext"))

## # A tibble: 1,704 x 9
##   dir      name ext  country      continent  year lifeExp      pop gdpPercap
##   <chr>    <chr> <chr> <chr>      <chr>      <dbl> <dbl>      <dbl> <dbl>
## 1 gapminder 1952  xlsx Afghanistan Asia      1952    28.8 8425333    779.
## 2 gapminder 1952  xlsx Albania     Europe    1952    55.2 1282697    1601.
## 3 gapminder 1952  xlsx Algeria     Africa    1952    43.1 9279525    2449.
## 4 gapminder 1952  xlsx Angola      Africa    1952    30.0 4232095    3521.
## 5 gapminder 1952  xlsx Argentina   Americas  1952    62.5 17876956    5911.
## 6 gapminder 1952  xlsx Australia   Oceania   1952    69.1 8691212    10040.
## 7 gapminder 1952  xlsx Austria     Europe    1952    66.8 6927772    6137.
## 8 gapminder 1952  xlsx Bahrain     Asia      1952    50.9 120447     9867.
## 9 gapminder 1952  xlsx Bangladesh   Asia      1952    37.5 46886859    684.
## 10 gapminder 1952  xlsx Belgium    Europe    1952    68   8730405    8343.
## # i 1,694 more rows
```

Saving

```
gapminder <- paths |>
  set_names() |>
  map(readxl::read_excel) |>
  list_rbind(names_to = "source") |>
  separate_wider_delim(source, delim = "/", names = c(NA, "dir", "file")) |>
  separate_wider_delim(file, delim = ".", names = c("name", "ext"))

write_csv(gapminder, "data/gapminder.csv")
```

Many simple iterations

Example of a function that reads in data from multiple files, filters out missing values, pivot, and the combines all the data:

```
process_file <- function(path) {
  df <- read_csv(path)

  df |>
    filter(!is.na(id)) |>
    mutate(id = tolower(id)) |>
    pivot_longer(jan:dec, names_to = "month")
}

paths |>
  map(process_file) |>
  list_rbind()
```

Alternative approach: perform each step of `process_file()` to every file:

```
paths |>
  map(read_csv) |>
  map(\(df) df |> filter(!is.na(id))) |>
  map(\(df) df |> mutate(id = tolower(id))) |>
  map(\(df) df |> pivot_longer(jan:dec, names_to = "month")) |>
  list_rbind()
```

The second approach is better because it stops the user from getting fixated on getting the first file right before moving on to the rest.

Another approach is to bind all the data frames together before doing the other operations:

```
paths |>
  map(read_csv) |>
  list_rbind() |>
  filter(!is.na(id)) |>
  mutate(id = tolower(id)) |>
  pivot_longer(jan:dec, names_to = "month")
```

Heterogeneous data

Sometimes, it is not possible to go from `map()` straight to `list_rbind()` because the data frames are so heterogeneous that `list_rbind()` either fails or yields a data frame that is not very useful.

A solution to problems like this usually involves loading all the files first and then capturing the structure of the data frames when you explore the resulting data:

```
df_types <- function(df) {
  tibble(
    col_name = names(df),
    col_type = map_chr(df, vctrs::vec_ptype_full),
    n_miss = map_int(df, \(x) sum(is.na(x)))
  )
}

df_types(gapminder)
```

```
## # A tibble: 9 x 3
##   col_name col_type n_miss
##   <chr>    <chr>    <int>
## 1 dir      character    0
## 2 name     character    0
## 3 ext      character    0
## 4 country  character    0
## 5 continent character    0
## 6 year     double      0
## 7 lifeExp  double      0
## 8 pop      double      0
## 9 gdpPercap double      0
```

Then the function can be applied to all the files:

```
files |>
  map(df_types) |>
  list_rbind(names_to = "file_name") |>
  select(-n_miss) |>
  pivot_wider(names_from = col_name, values_from = col_type)

## # A tibble: 12 x 7
##   file_name country continent year lifeExp pop gdpPercap
##   <chr>    <chr>    <chr>    <chr> <chr>  <chr>  <chr>
## 1 1952.xlsx character character double double double double
## 2 1957.xlsx character character double double double double
## 3 1962.xlsx character character double double double double
## 4 1967.xlsx character character double double double double
## 5 1972.xlsx character character double double double double
## 6 1977.xlsx character character double double double double
## 7 1982.xlsx character character double double double double
## 8 1987.xlsx character character double double double double
## 9 1992.xlsx character character double double double double
## 10 1997.xlsx character character double double double double
## 11 2002.xlsx character character double double double double
## 12 2007.xlsx character character double double double double
```

If the files have heterogeneous formats, you might need to do more processing before you can successfully merge them.

Handling failures

One of the downsides of `map()`: it either succeeds or fails as a whole. Solution: The `purrr` package called `possibly()`.

Using `possibly()` to tackle an erring function:

```

files <- paths |>
  map(possibly(\(path) readxl::read_excel(path), NULL))

# testing
data <- files |> list_rbind()

data

## # A tibble: 1,704 x 6
##   country      continent year lifeExp      pop gdpPercap
##   <chr>         <chr>    <dbl>  <dbl>    <dbl>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Albania      Europe    1952   55.2  1282697   1601.
## 3 Algeria      Africa    1952   43.1  9279525   2449.
## 4 Angola       Africa    1952   30.0  4232095   3521.
## 5 Argentina    Americas  1952   62.5 17876956   5911.
## 6 Australia    Oceania   1952   69.1  8691212  10040.
## 7 Austria      Europe    1952   66.8  6927772   6137.
## 8 Bahrain      Asia      1952   50.9   120447   9867.
## 9 Bangladesh   Asia      1952   37.5 46886859    684.
## 10 Belgium     Europe    1952    68   8730405   8343.
## # i 1,694 more rows

# trying the function on something that will fail
failed <- map_vec(files, is.null)
paths[failed]

```

```
## character(0)
```

The function can be called again for each failure and figure out what went wrong.

Saving multiple outputs

How to take one or more R objects and save it to one or more files?

Writing to a database

Building the data “by hand” Creating a template by reading a single file and adding more files to it:

```

template <- readxl::read_excel(paths[1])
template$year <- 1952
template

```

```

## # A tibble: 142 x 6
##   country      continent year lifeExp      pop gdpPercap
##   <chr>         <chr>    <dbl>  <dbl>    <dbl>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Albania      Europe    1952   55.2  1282697   1601.
## 3 Algeria      Africa    1952   43.1  9279525   2449.
## 4 Angola       Africa    1952   30.0  4232095   3521.
## 5 Argentina    Americas  1952   62.5 17876956   5911.
## 6 Australia    Oceania   1952   69.1  8691212  10040.
## 7 Austria      Europe    1952   66.8  6927772   6137.
## 8 Bahrain      Asia      1952   50.9   120447   9867.
## 9 Bangladesh   Asia      1952   37.5 46886859    684.
## 10 Belgium     Europe    1952    68   8730405   8343.

```

```
## # i 132 more rows
```

Using `DBI::dbCreateTable()` to turn the template into a database table:

```
con <- DBI::dbConnect(duckdb::duckdb())
DBI::dbCreateTable(con, "gapminder", template)
```

Inspecting the newly-created db table:

```
con |> tbl("gapminder")
```

```
## # Source:   table<gapminder> [?? x 6]
## # Database: DuckDB v1.3.2 [norman@Linux 6.16.1-2-cachyos:R 4.5.1/:memory:]
## # i 6 variables: country <chr>, continent <chr>, year <dbl>, lifeExp <dbl>,
## #   pop <dbl>, gdpPercap <dbl>
```

Creating a function that takes a single filepath, reads it into R, and then adds the result to the `gapminder` table.

```
append_file <- function(path) {
  df <- readxl::read_excel(path)
  df$year <- parse_number(basename(path))

  DBI::dbAppendTable(con, "gapminder", df)
}
```

One possible code for appending the files for each filepath in `paths`:

```
paths |> map(append_file)
```

Better way: Use `walk()` instead of `map()`. The latter does exactly the same thing as `map()` but throws the output away:

```
paths |> walk(append_file)
```

Checking to see if all the data are appended to the table:

```
con |>
  tbl("gapminder") |>
  count(year)
```

```
## # Source:   SQL [?? x 2]
## # Database: DuckDB v1.3.2 [norman@Linux 6.16.1-2-cachyos:R 4.5.1/:memory:]
##   year      n
##   <dbl> <dbl>
## 1  1977    142
## 2  1987    142
## 3  1967    142
## 4  1962    142
## 5  1982    142
## 6  1952    142
## 7  1957    142
## 8  1972    142
## 9  1992    142
## 10 1997    142
## 11 2002    142
## 12 2007    142
```


Writing .csv files

Imagine that we want to take the `ggplot2::diamonds` data and save one csv file for each clarity.

There are many ways to skin the cat, but the authors recommend using `group_nest()`:

```
by_clarity <- diamonds |>
  group_nest(clarity)
```

```
by_clarity
```

```
## # A tibble: 8 x 2
##   clarity      data
##   <ord>    <list<tibble[,9]>>
## 1 I1      [741 x 9]
## 2 SI2     [9,194 x 9]
## 3 SI1     [13,065 x 9]
## 4 VS2     [12,258 x 9]
## 5 VS1     [8,171 x 9]
## 6 VVS2     [5,066 x 9]
## 7 VVS1     [3,655 x 9]
## 8 IF      [1,790 x 9]
```

The data column is a list-column that contains one tibble for each unique value of `clarity`:

```
by_clarity$data[[1]]
```

```
## # A tibble: 741 x 9
##   carat cut      color depth table price      x      y      z
##   <dbl> <ord>    <ord> <dbl> <dbl> <int> <dbl> <dbl> <dbl>
## 1  0.32 Premium E      60.9   58   345  4.38  4.42  2.68
## 2  1.17 Very Good J      60.2   61  2774  6.83  6.9   4.13
## 3  1.01 Premium F      61.8   60  2781  6.39  6.36  3.94
## 4  1.01 Fair   E      64.5   58  2788  6.29  6.21  4.03
## 5  0.96 Ideal  F      60.7   55  2801  6.37  6.41  3.88
## 6  1.04 Premium G      62.2   58  2801  6.46  6.41   4
## 7  1 Fair     G      66.4   59  2808  6.16  6.09  4.07
## 8  1.2 Fair   F      64.6   56  2809  6.73  6.66  4.33
## 9  0.43 Very Good E      58.4   62   555  4.94  5     2.9
## 10 1.02 Premium G      60.3   58  2815  6.55  6.5   3.94
## # i 731 more rows
```

Adding a new column to `by_clarity`, to inform us of which filename to assign to each tibble:

```
by_clarity <- by_clarity |>
  mutate(path = str_glue("diamonds-{clarity}.csv"))
```

```
by_clarity
```

```
## # A tibble: 8 x 3
##   clarity      data path
##   <ord>    <list<tibble[,9]>> <glue>
## 1 I1      [741 x 9] diamonds-I1.csv
## 2 SI2     [9,194 x 9] diamonds-SI2.csv
## 3 SI1     [13,065 x 9] diamonds-SI1.csv
## 4 VS2     [12,258 x 9] diamonds-VS2.csv
## 5 VS1     [8,171 x 9] diamonds-VS1.csv
## 6 VVS2     [5,066 x 9] diamonds-VVS2.csv
```

```
## 7 VVS1          [3,655 x 9] diamonds-VVS1.csv
## 8 IF            [1,790 x 9] diamonds-IF.csv
```

In saving the files “by hand,” we need to write something like this:

```
write_csv(by_clarity$data[[1]], by_clarity$path[[1]])
write_csv(by_clarity$data[[2]], by_clarity$path[[2]])
...
```

Since there are two arguments that are changing here, the function `map()` will not work here. Instead, we can use `map2()`. However, because we don’t care to see the output, it is better to use `walk2()`.

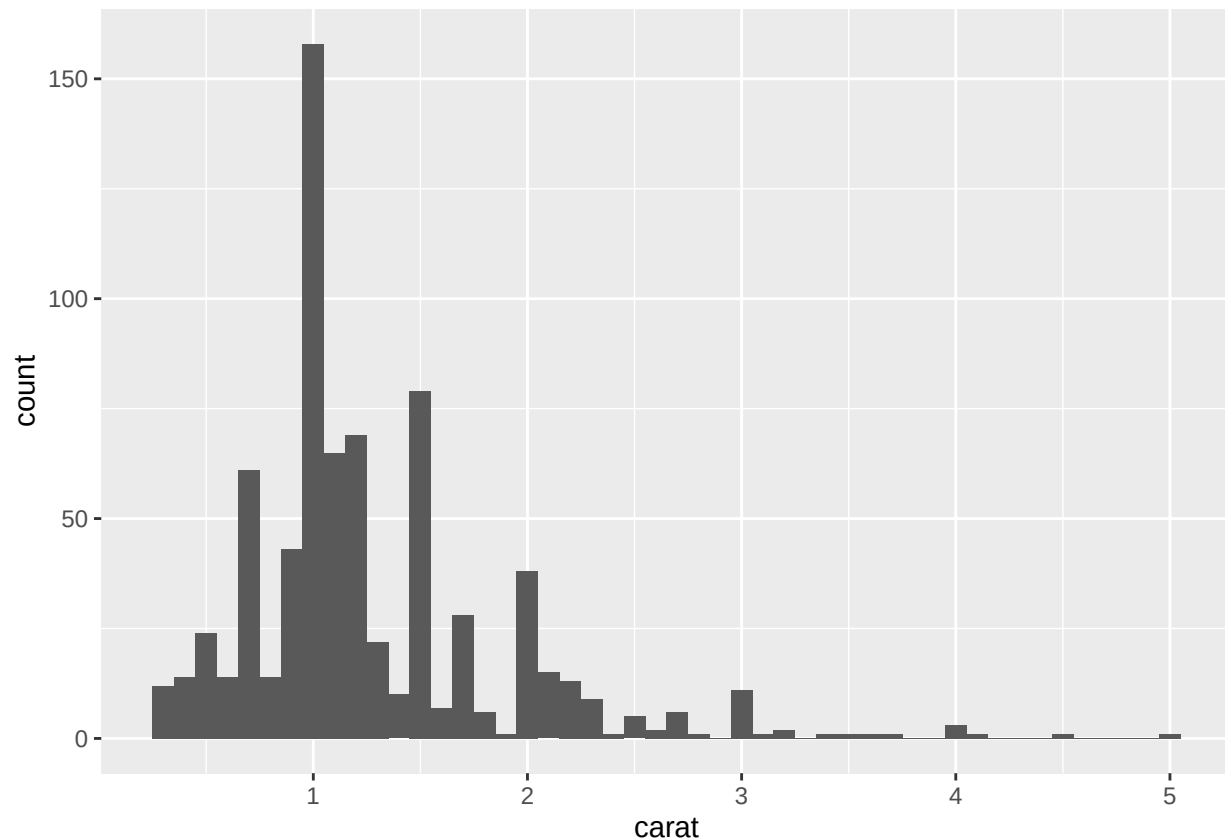
```
walk2(by_clarity$data, by_clarity$path, write_csv)
```

Saving plots

The same approach can be used to create many plots – just write the correct function:

```
carat_histogram <- function(df) {
  ggplot(df, aes(x = carat)) +
    geom_histogram(binwidth = 0.1)
}

carat_histogram(by_clarity$data[[1]])
```



Now we can use `map()` to create a list of many plots and their eventual filepaths:

```
by_clarity <- by_clarity |>
  mutate(
    plot = map(data, carat_histogram),
```

```

    path = str_glue("clarity-{clarity}.png")
  )

by_clarity

## # A tibble: 8 x 4
##   clarity      data path      plot
##   <ord>    <list<tibble[,9]>> <glue>    <list>
## 1 I1      [741 x 9] clarity-I1.png <gg>
## 2 SI2     [9,194 x 9] clarity-SI2.png <gg>
## 3 SI1    [13,065 x 9] clarity-SI1.png <gg>
## 4 VS2    [12,258 x 9] clarity-VS2.png <gg>
## 5 VS1     [8,171 x 9] clarity-VS1.png <gg>
## 6 VVS2     [5,066 x 9] clarity-VVS2.png <gg>
## 7 VVS1     [3,655 x 9] clarity-VVS1.png <gg>
## 8 IF      [1,790 x 9] clarity-IF.png   <gg>

```

Finally, we can use `walk2()` again, this time, with `ggsave()` to save each plot:

```

walk2(
  by_clarity$path,
  by_clarity$plot,
  \(path, plot) ggsave(path, plot, width = 6, height = 6)
)

```